4차 프로젝트 계획 정리

SKN35 1팀 · 3차 Docentalk 고도화 · 2026-11-03

> **한 줄 요약**
> 4차는 3차 Docentalk을 이어서 발전시키는 프로젝트예요. 할 일은 **Django로 백엔드 옮기기**, **LangGraph로 답변 흐름 정리**, **하이브리드 검색으로 낮았던 점수 올리기**, **AWS·Docker·CI/CD로 배포하기**예요. 프론트는 3차에서 배포한 Flutter Web을 그대로 써요.

주제: LLM을 연동한 내외부 문서 기반 질의응답 웹페이지 개발
필수 산출물: 요구사항 정의서, 화면설계서, LLM 연동 웹 애플리케이션, 시스템 구성도, 테스트 계획 및 결과 보고서

`팀원 PDF` 표시는 팀원이 정리한 「아키텍처 구조 → 보완해야 할 점」에서 가져온 내용이에요.

1. 해야 할 일

3차와 4차가 어디서 다른지부터 보면 할 일이 정리돼요.

항목	3차	4차	할 일
백엔드	FastAPI (Railway)	**Django** (+ Django Ninja)	API 틀만 새로 쓰고, RAG·서비스 로직은 옮겨 쓰기
프론트	Flutter Web (Vercel)	Flutter Web 유지	API 주소와 응답 모양은 3차와 같게 유지. 빌드 결과물을 nginx로 서빙
답변 흐름	if문 + 같은 흐름이 두 곳에 있음	**LangGraph**	흐름을 그래프 하나로 합치기
배포	Railway · Vercel	**AWS EC2 + Docker**	직접 서버 구성, nginx, HTTPS, CI/CD
문서	외부 미술 자료만 있음	내부 + 외부 문서	(선택) 사용자 PDF 업로드 질의응답

① 기획 · 요구사항 정의서

- **범위 정하기**: 3차 기능(소셜, 일기, MCP 등)을 다 옮기면 일이 너무 많아요. 핵심인 **도슨트 채팅, 출처 표시, 작품 목록·상세, 로그인, 컬렉션**을 먼저 하고, 나머지는 여유가 있으면 해요.
- 기능·비기능 요구사항에 ID(FR-01, NFR-01 …)를 붙이고 우선순위를 매겨요. 3차 README의 기능 표와 `PLAN.md`에서 시작하면 돼요.
- 4차에서 새로 넣는 것(LangGraph, 하이브리드 검색, AWS 배포)은 비기능 요구사항(성능·정확도·가용성)으로 정리해요.

② 화면설계서

- 프론트는 3차 Flutter Web을 그대로 써요. 3차 운영 캡처(`docs/readme-2026-10-15/`)를 기반으로 정리하면 돼요.
- 화면마다 화면 ID, 구성 요소, 동작, 연결 API를 적어요.
- 새 화면이 생기면(예: 문서 업로드) 그 화면만 새로 그려요.

③ Django 백엔드

- 기능별로 앱을 나눠요: `accounts`, `catalog`, `chat`, `library`, `documents`, `llm`
- **Django Ninja**를 쓰면 FastAPI와 문법이 거의 같아서 `api/routes/`를 쉽게 옮길 수 있어요. 다른 기수도 이 방식을 썼어요.
- SQLAlchemy 모델(`models/`)을 Django ORM으로, Alembic을 Django migration으로 바꿔요. 지금 TiDB 테이블을 그대로 쓰려면 `inspectdb`로 모델을 뽑아 오면 빨라요.
- `rag/`, `docent/`, `clients/`, `services/`는 거의 그대로 옮겨요. DB 접근만 Django ORM에 맞게 고쳐요.
- 스트리밍 답변(SSE)은 `StreamingHttpResponse`로 유지해요. 이벤트 모양(`token`, `replace`, `done`)은 3차와 같게 둬요.
- Django Admin으로 작품·문서를 관리하면 관리자 화면을 따로 만들지 않아도 돼요.

④ 프론트 (Flutter Web)

- 3차에서 실제로 배포한 Flutter Web(`lib/`, `web/`, `assets/`, `pubspec.yaml`)을 그대로 가져와요. 3차의 React(`src/`)는 API 연결이 없는 초기 목업이라 쓰지 않아요.
- 바꿀 것은 빌드할 때 넣는 `API_BASE_URL` 정도예요(`scripts/write-build-defines.mjs`). `flutter build web` 결과물(`build/web`)을 nginx가 서빙해요.
- Flutter가 Firebase 로그인을 쓰니까, Django에서도 Firebase ID 토큰 검증(`firebase-admin`)을 3차처럼 유지해요.

⑤ 클라우드 인프라 · 배포 · 시스템 구성도

- **AWS EC2 + Docker Compose**로 구성해요: nginx(HTTPS, 정적 파일) → Gunicorn/Uvicorn(Django) → TiDB · Chroma Cloud · OpenAI
- GitHub Actions로 CI/CD를 만들어요: push하면 테스트(CI), main에 merge하면 EC2에 자동 배포(CD)
- 비밀값은 `.env.example`만 커밋하고, 실제 값은 GitHub Secrets와 EC2 환경변수에 넣어요.
- 시스템 구성도는 3차 README의 mermaid 구조도를 4차 구성(AWS, LangGraph)에 맞게 다시 그려요.

⑥ 테스트 계획 및 결과 보고서

- 3차 평가 자료를 그대로 써요: `tests/questions/*.json`(45문항), `scripts/eval_*.py`, RAGAS 기반 LLM Judge
- 추가할 것: Django API 테스트, 배포 환경 통합 테스트, 화면 시나리오 체크리스트
- 보고서는 **3차 기준 → 4차 개선 후** 비교표로 만들어요. 이 문서의 4장 내용이 그대로 들어가요.

2. 확장 — LangGraph · 고도화

지금 상태

LangChain은 이미 쓰고 있어요(`ChatOpenAI`, `OpenAIEmbeddings`, `JsonOutputParser`, `Document`). LangGraph는 안 쓰고 있어요.
LangChain은 부품이고, LangGraph는 부품을 어떤 순서로 실행할지 정하는 설계도예요.

지금은 `docent/service.py`의 `answer()` 안에 if문으로 흐름이 짜여 있어요.

```
질문 → 분류 → 거절? → 바로 답변? → 범위 밖? → 추천? → 검색 → 자료 없음? → LLM 생성 → 검증 → 답변
```

팀원 PDF 게다가 **같은 흐름이 두 곳에 따로 구현돼 있어요.** `docent_service.answer`(일반 답변)와 `docent_stream.answer_events`(스트리밍)가 분류·범위 판정·검색·생성·검증을 각각 해요. 스트리밍 쪽은 RAG가 아닌 질문을 처리할 때 `answer()`를 다시 불러서 분류를 한 번 더 해요. 그래서 한쪽만 고치면 두 경로의 동작이 달라져요.
→ **LangGraph로 그래프 하나를 만들고, 일반 답변과 스트리밍이 같은 그래프를 쓰게 하면** 이 문제가 구조적으로 해결돼요. LangGraph를 써야 하는 가장 분명한 이유예요.

① 기본: 지금 흐름을 그래프로 옮기기

```
[분류] ─거절──→ [거절 답변] ──→ 끝
   ├──바로 답변→ [카탈로그 답변] ──→ 끝
   ├──추천──────→ [추천] ──→ 끝
   └──RAG───────→ [검색] → [생성] → [검증] ──→ 끝
```

- 지금 있는 `classify`, `retrieve`, `generate`, `validate_response`를 노드로 감싸기만 하면 돼요. 새로 짜는 로직은 거의 없어요.
- 흐름도 그림이 자동으로 나와서 시스템 구성도와 발표 자료에 바로 쓸 수 있어요.
- 스트리밍은 LangGraph의 `stream_mode="messages"`로 생성 노드의 토큰을 받아요. 그래서 흐름을 한 번만 짜면 돼요.

② 고도화: "근거가 부족하면 다시 찾기" 루프

```
[검색] → [근거 판정] ─부족→ [질문 다시 쓰기(LLM)] → [검색]   (최대 2회)
                    ├──충분→ [생성]
                    └──2회 후에도 부족→ [거절(NO_EVIDENCE)]  ← LLM 호출 없이
```

- 팀원 PDF 지금 후속 질문 처리는 "그 사람", "아까" 같은 말을 정규식으로 찾아서 직전 질문을 앞에 붙이는 방식이에요. 이걸 **LLM으로 질문을 다시 쓰는 노드**로 바꿔요.
- 팀원 PDF **관련도 기준선**: 지금은 거리가 아무리 멀어도 검색 결과가 항상 LLM에 들어가요. 근거 판정 노드에서 점수가 기준 미만이면 LLM을 부르지 않고 바로 거절해요. 기준값은 기존 평가 세트에서 정답이 있는 질문과 거절해야 하는 질문의 점수 분포를 비교해서 정해요.

③ 확장: 대화 기억 (checkpointer)

- LangGraph checkpointer로 이전 대화 맥락을 이어받아요. 지금 DB에 저장하는 대화 기록과 연결해요.
- 지금은 "최근 6턴 × 600자"처럼 글자 수로 자르는데, 이걸 토큰 기준으로 바꾸는 작업과 같이 하면 좋아요(3장 참고).

④ 그 밖의 확장 (선택)

기능	내용	효과
내부 문서 업로드	사용자가 PDF(전시 도록, 감상 노트 등)를 올리면 텍스트 추출 → 청킹 → 임베딩 → 그 문서로도 질문하기	주제 "**내외부 문서** 기반 질의응답"과 정확히 맞아요
LangSmith 추적	노드별 걸린 시간과 입출력 기록	테스트 보고서와 디버깅에 좋아요
Semantic Cache	비슷한 질문은 저장된 답을 돌려주기	응답 속도(P95 5.9초) 개선
DB 백업	S3 자동 백업	운영 안정성

3. 보완할 점

인프라 · 구조

영역	지금 상태	보완 방향	우선순위
배포	Railway + Vercel	AWS EC2 + Docker Compose + nginx + HTTPS	⭐⭐⭐ 필수
CI/CD	없음	GitHub Actions: 자동 테스트 후 자동 배포	⭐⭐⭐
백엔드	FastAPI	Django + Django Ninja	⭐⭐⭐ 필수
파이프라인 중복 팀원 PDF	일반 답변과 스트리밍에 흐름이 따로 있음	LangGraph 그래프 하나로 통합 (2장 ①)	⭐⭐⭐
범위 밖 질문을 세 번 거름 팀원 PDF	정규식 분류기, LLM 범위 판정, 시스템 프롬프트 거절 규칙이 같은 일을 겹쳐서 해요. 그래서 잘못 걸렀을 때 어디를 고쳐야 할지 알기 어려워요	범위 판정을 그래프의 한 노드로 모으고, 어느 단계에서 걸렀는지 응답 로그에 남기기	⭐⭐
규칙 하드코딩 팀원 PDF	`OUT_OF_SCOPE_ARTISTS`, 비교 키워드, 후속 질문 정규식이 코드 안에 있음. 별칭은 `lru_cache`라서 데이터가 바뀌면 서버를 재시작해야 함	규칙을 `data/`의 설정 파일이나 DB로 옮기기(Django Admin에서 수정 가능). 캐시는 데이터 갱신 시점에 비우기	⭐⭐
관측·모니터링	없음	LangSmith, 헬스체크	⭐⭐

검색 품질 팀원 PDF

문제	보완 방향
벡터 검색만 씀: 키워드 검색(BM25)이나 리랭커가 없어서 고유명사, 연도, 작품명처럼 정확한 단어를 맞춰야 하는 질문에 약해요	하이브리드 검색 (아래「하이브리드 RAG 설계」)
관련도 기준선 없음: 거리가 멀어도 항상 LLM에 넣고, 거절 판단은 전부 LLM이 해요	점수 기준 미만이면 LLM 호출 없이 거절 (2장 ②)
질문 다시 쓰기가 단순함: 정규식으로 직전 질문을 앞에 붙이는 방식	LLM으로 질문 다시 쓰기 (2장 ②)
Chroma 우회 비용이 큼: 필터를 건 근사 검색에서 청크가 빠지는 문제를 피하려고 `get()`으로 벡터를 전부 받아 numpy로 계산해요. 문서가 늘면 요청마다 받는 양도 늘어요	작품별 청크를 메모리 TTL 캐시에 두기 (아래 설계)
TiDB와 Chroma를 손으로 동기화: `sync_chroma`를 직접 돌려야 맞춰지고, 어긋났는지 알 방법이 없어요	문서 수·청크 ID를 비교하는 정합성 검사 스크립트를 만들고, CI나 헬스체크에서 돌리기

생성 · 검증 팀원 PDF

문제	보완 방향

출처를 문서 단위로만 확인: `used_document_ids`가 실제 검색된 문서인지만 보고, 답변의 각 주장이 근거와 맞는지는 확인하지 않아요	(선택) 검증 노드에서 문장별로 근거와 맞는지 LLM Judge로 확인. 비용이 드니까 평가 단계에서 먼저 써 보기
스트리밍에서 검증 전 문장이 먼저 보임: 검증에 실패하면 `replace` 이벤트로 거절 문구로 바뀌어서, 근거 없는 문장이 잠깐 보였다가 화면이 깜빡여요	2장 ②의 관련도 기준선으로 실패할 질문은 생성 전에 걸러서 `replace`가 생기는 경우 자체를 줄이기. 필요하면 첫 문장만 버퍼링
컨텍스트를 글자 수로만 자름: 문서 최대 40개, 문서당 5000자, 이력 6턴 × 600자	`tiktoken`으로 토큰 예산을 정해서 자르기

하이브리드 RAG 설계 팀원 PDF

팀원이 제안한 A안이에요. 지금 데이터 규모에 가장 단순하고 빨라요.

1. **작품별 청크를 메모리 캐시에 두기**: 작품별로 범위 안 청크의 본문과 벡터를 한 번 불러와 TTL 캐시에 둬요(키: `artwork_id`). `sync_chroma`나 `build_chunks`를 돌리면 캐시를 비워요.
   → Chroma `query()` / `get()` 왕복이 0회가 되고, 필터 검색에서 청크가 빠지는 문제가 사라져요. 남는 외부 호출은 질문 임베딩 1회뿐이에요.
2. **한국어 BM25**: `kiwipiepy` 형태소 분석으로 명사·고유명사·숫자만 색인하고, `rank_bm25`를 써요. 작가 별칭(`artist_aliases.jsonl`)을 사용자 사전에 넣어서 "렘브란트 판 레인" 같은 이름이 쪼개지지 않게 해요.
   → 연도(1642), 작품명(『야경』), 기법명(테네브리즘) 같은 질문이 좋아져요.
3. **RRF로 합치기**: `점수 = 1/(60+벡터순위) + 1/(60+BM25순위)`. 두 점수의 단위를 맞출 필요가 없어서 튜닝이 쉬워요. 지금의 자리 보장 규칙(`MAX_PER_DOC`, `OWN_K`, 연표·보정문 고정)은 RRF 뒤에 그대로 적용해요.
4. **리랭커는 선택**: RRF 상위 15~20개만 리랭크해서 5~8개로 줄여요. Cohere `rerank-multilingual`(API, 수백 ms)이나 `bge-reranker-v2-m3`(직접 호스팅, CPU·메모리 부담 큼) 중에서 골라요. 1~3번을 먼저 적용해서 평가해 보고, 점수가 부족할 때만 추가해요.
5. **관련도 기준선으로 일찍 거절**: 최상위 점수가 기준 미만이면 `NO_EVIDENCE_MESSAGE`로 바로 거절해요.
6. **대안**: 문서가 크게 늘면 TiDB 전문 검색(FULLTEXT) + 벡터 검색, 또는 Chroma Cloud sparse 벡터로 옮길 수 있어요. 지금 규모에서는 1번이 더 빠르고 단순해요.

4. 테스트 기준 미달 항목과 해결 방법

❌ ① `refused` 값 정합성 6/8 (75%)

- **무슨 문제냐면**: 답변 본문에는 "이 자료로는 확인할 수 없어요"라고 쓰고, 거절 여부 값은 `refused=false`로 보낸 경우예요. 내용이 틀린 게 아니라 표시가 틀린 거예요.
- **원인**: `docent/grounding.py`에서 LLM이 `refused` 값을 직접 정하는데, 가끔 본문과 값이 엇갈려요.
- **해결** (쉬운 순서)
  1. **서버에서 한 번 더 확인**: 출처(`used_document_ids`)가 하나도 없거나 답변에 "확인할 수 없", "자료에 없" 같은 표현이 있으면 서버가 `refused=true`로 고쳐요. 코드 몇 줄이면 돼요.
  2. **프롬프트 보강**: "확인 불가를 설명할 때는 반드시 refused=true" 규칙과 예시를 넣어요.
  3. LangGraph의 **[검증] 노드**에 1번 로직을 넣어요.
- 가장 빨리 고칠 수 있어요. 보고서에 "75% → 100%"로 바로 쓸 수 있어요.

❌ ② 청크 단위 검색 점수 미달

지표	3차 결과	목표
Recall@5	0.787	0.85
MRR	0.778	0.80
nDCG@5	0.751	0.80

- **무슨 문제냐면**: 문서 단위로는 100% 찾아요. 그런데 **문서 안의 정확한 문단(청크)**을 상위 5개에 넣는 건 조금 부족해요. 정답 문단이 6~10위에 있거나 순서가 뒤로 밀려 있어요.
- **원인**: 벡터 검색 하나만 써서 "인물 배치", "명암", 연도, 작품명 같은 특정 단어가 들어간 문단을 놓쳐요. 팀원 PDF에서 짚은 "벡터 검색만 씀" 문제와 같은 원인이에요.
- **해결** (효과 큰 순서)
    1. **하이브리드 검색**: 한국어 BM25 + 벡터를 RRF로 합쳐요(3장 설계 1~3번).
    2. **Rerank**: 후보를 15~20개 가져와서 재정렬해요. 순서 점수인 **MRR과 nDCG를 올리는 데 가장 효과적**이에요.
    3. **질문 다시 쓰기**: LangGraph ② 루프예요.
    4. **청킹 조정**: 지금은 500자(최대 800자), 150자 미만은 합쳐요. 크기나 겹침을 바꿔 볼 수 있지만, 효과가 불확실하고 다시 임베딩해야 해서 마지막에 해요.
- **보고서 형식**: 단계별 비교표로 만들어요. 평가 스크립트(`scripts/eval_retrieval.py`)와 문항(`tests/questions/chunk-ground-truth.json`)이 있어서 그대로 다시 돌리면 돼요.

단계	Recall@5	MRR	nDCG@5
3차 (벡터만)	0.787	0.778	0.751
+ BM25 하이브리드 (RRF)			
+ Rerank			
+ 질문 다시 쓰기			

⚠️ 그 밖에 남은 것 (3차 README의 후속 과제)

- 출처 링크 1개 수동 확인 → 직접 열어 보고 교체
- 실제 두 계정으로 저장·삭제·권한 격리 검증 → 테스트 계정 2개로 시나리오 테스트
- 브라우저에서 첫 글자가 뜨기까지 시간 측정 → 배포 후 측정해서 보고서에 넣기
- 4차 추가: 관련도 기준선을 넣은 뒤 **거절해야 할 질문은 거절하고, 답해야 할 질문은 거절하지 않는지** 같이 측정 (거절을 너무 많이 하지 않는지)

5. 진행 순서와 폴더 구조

```
1주차  요구사항 · 범위 확정 · Django 뼈대 · 3차 코드 이식 · refused 수정
2주차  LangGraph 그래프(①) · 하이브리드 검색 · 평가 다시 돌리기
3주차  LangGraph 루프(②) · Rerank · Docker/AWS 배포 · CI/CD
4주차  (선택) 내부 문서 업로드 · 테스트 보고서 · 시스템 구성도 · 발표
```

```
SKN35-4th-1Team/
├── backend/                # Django
│   ├── config/              # settings, urls, api (Django Ninja)
│   ├── accounts/            # 회원 · 로그인
│   ├── catalog/             # 작가 · 작품 · 사조
│   ├── chat/                # 도슨트 채팅 API
│   ├── library/             # 좋아요 · 컬렉션
│   ├── documents/           # 외부 자료 + 내부 문서 업로드
│   ├── llm/                 # 3차 rag/ + docent/ + clients/ 이식 → LangGraph
│   ├── scripts/             # 전처리 · 임베딩 · 평가 (3차 복사)
│   ├── tests/               # 평가 문항 · 단위 테스트 (3차 복사)
│   └── manage.py
├── frontend/               # Flutter Web (3차 복사)
├── data/                   # 3차 복사
├── infra/                  # Dockerfile, docker-compose, nginx
├── .github/workflows/      # CI/CD
└── docs/                   # 4차 산출물
```

3차 레포: SKN35-3rd-1Team · 팀원 자료:「(3차 단프-1팀) 아키텍처 구조 → 보완해야 할 점」